التوثيق التفصيلي للنظام الأكاديمي متعدد الوكلاء

هذا المستند يشرح كل خطوة تم تنفيذها في المشروع، مع توضيح الهدف الهندسي من كل قرار تقني، بدون عناصر عاطفية أو ترويجية.

---

1) تعريف المشكلة

المشكلة الأساسية:

الطلاب يحتاجون نظامًا يساعدهم في:

- فهم اللوائح الأكاديمية
- تخطيط المقررات
- حساب المعدل
- تقييم المخاطر الأكاديمية
- الاحتفاظ بسياق طويل للمحادثة

الحل المقترح:
بناء نظام ذكاء اصطناعي يعتمد على:

- RAG لاسترجاع اللوائح
- ذاكرة طويلة المدى لكل مستخدم
- واجهة تفاعلية
- قاعدة بيانات منظمة

---

2) تصميم قاعدة البيانات

تم إنشاء قاعدة بيانات على Supabase.

السبب:

- توفر PostgreSQL
- دعم pgvector
- دعم Row Level Security
- تكامل مع Auth

الجداول الأساسية

1. universities
   لتجهيز النظام ليكون قابلًا لدعم عدة جامعات مستقبلاً.
2. faculties / programs / courses
   لبناء هيكل أكاديمي منظم.
3. student_courses
   لتخزين مقررات الطالب وحالته الأكاديمية.

4. documents + document_chunks
لتخزين لوائح الجامعة على شكل أجزاء قابلة للبحث المتجهي.
5. user_memory
لتخزين الذاكرة طويلة المدى لكل مستخدم.
6. conversations + messages
لتخزين المحادثات بشكل دائم.

---

3) RLS (Row Level Security)

تم تفعيل RLS.

السبب:

- منع الطالب من رؤية بيانات غيره
- السماح للـ Admin برؤية بيانات الجامعة
- ضمان العزل بين المستخدمين

هذا ضروري لأي نظام متعدد المستخدمين.

---

4) بناء RAG

RAG = Retrieval Augmented Generation

الخطوات:

1. تقسيم اللوائح إلى أجزاء صغيرة (Chunks)
السبب: تحسين دقة الاسترجاع وتقليل الضوضاء.
2. توليد Embeddings لكل جزء
السبب: تمكين البحث الدلالي بدلاً من البحث النصي التقليدي.
3. تخزين embeddings في document_chunks
باستخدام pgvector.
4. إنشاء RPC باسم match_documents
السبب: تنفيذ بحث متجهي فعال داخل PostgreSQL.
5. عند كل سؤال:

- حساب embedding للسؤال
- استرجاع أفضل k نتائج
- دمجها في Prompt

الهدف:
منع النموذج من الإجابة من المعرفة العامة فقط.

---

5) إضافة الذاكرة طويلة المدى

المشكلة:
RAGوحده لا يتذكر سياق الطالب السابق.

الحل:
بناء جدول.user_memory

الخطوات:

1. بعد كل رسالة:

- حسابembedding
- تخزينها معuser_id

2. عند سؤال جديد:

- حساب embedding للسؤال
- استرجاع أكثر الذكريات صلة
- إدراجها داخلPrompt

الهدف:
تمكين النظام من تخصيص الإجابات.

ملاحظة:
هذه ليست ذاكرة داخل.LLM
بل ذاكرة خارجية تعتمد على.Vector Search

---

6) تصميمPrompt

تم تصميم Prompt يحتوي على ثلاثة أقسام:

1. User Memory
2. Regulation Context
3. Student Question

مع قواعد واضحة:

- اللوائح هي المصدر الأساسي
- الذاكرة فقط للتخصيص
- في حال عدم وجود معلومة يتم التصريح بذلك

الهدف:
تقليل الهلوسة.

---

7) Backend Architecture

تم استخدام FastAPI.

السبب:

- سرعة الأداء
- دعم async
- مناسب لواجهات REST

تم فصل النظام إلى:

- agents
- services
- core

الهدف:
تنظيم الكود وتقليل الترابط.

---

8) Embedding Strategy

تم استخدام نموذج:
sentence-transformers/paraphrase-multilingual-mpnet-base-v2

ثم تم تقييم خيارين:

1. Local Model
مزاياه:

- لا يعتمد على API خارجي

عيوبه:

- يستهلك RAM
- يزيد حجم Docker

2. HuggingFace API
مزاياه:

- لا يستهلك موارد محلية
- مناسب للنشر السريع

تم توضيح أن الإنتاج الحقيقي يفضل استخدام Embedding API سحابي.

---

9) Frontend

تم بناء الواجهة باستخدام:
Next.js + Framer Motion + Supabase

الوظائف:

- Dashboard
- Chat Page
- Auth
- Persistent Conversations

عند فتح الصفحة:

- يتم التأكد من المستخدم
- يتم تحميل آخرConversation
- يتم تحميل الرسائل

عند إرسال رسالة:

- تخزين رسالة المستخدم
- إرسال الطلب إلىBackend
- تخزين رد الذكاء الاصطناعي

---

10) Orchestration Flow

تسلسل التنفيذ:

1. استقبال السؤال
2. استرجاع سياق اللوائح
3. استرجاع الذاكرة طويلة المدى
4. دمج السياقات فيPrompt
5. استدعاءLLM
6. تخزين الذاكرة الجديدة
7. إعادة الرد إلى الواجهة

الهدف:
فصل المهام بدلاً من الاعتماد على استدعاء واحد مباشر.

---

11) قابلية التوسع

النظام حالياً لجامعة واحدة.

لكن التصميم يدعم:

- إضافة جامعات متعددة
- Multi-Tenant Architecture
- فصل البيانات حسبuniversity_id

---

12) الحالة الحالية للنظام

تم تنفيذ:

- قاعدة بيانات كاملة
- RLS
- RAG
- ذاكرة طويلة المدى
- تخزين المحادثات
- Auth
- واجهة مستخدم تفاعلية

النظام يعمل كبنية معمارية متكاملة.

---

13) الخطوات المنطقية التالية

1. إضافةMulti-Conversation Sidebar
2. تحسين الذاكرة عبرSummarization
3. إضافةRisk Prediction Agent
4. إضافةPlanner Optimization Agent
5. بناء لوحة تحكم Admin تحليلية